

AKIENI ACADEMY – COHORTE 2

Projet Pratique – Semaines 14 & 15

Application de gestion d'une bibliothèque de quartier – Backend, Frontend & Base de données (projet sur 2 semaines)

Contexte du projet

Une bibliothèque de quartier souhaite informatiser la gestion de ses livres, de ses adhérents et de ses emprunts. Elle vous confie la conception et la réalisation complète de l'application : de la base de données jusqu'à l'interface utilisée par le personnel.

Ce projet est réparti sur **deux semaines** afin de vous laisser le temps de traiter un périmètre fonctionnel complet – gestion des adhérents, statistiques, recherche avancée – en plus du cœur du sujet (CRUD, authentification des emprunts, middlewares). Il mobilise l'ensemble des acquis du Module 3 (base de données relationnelle, SQL, Node.js, Express) ainsi que vos acquis des Modules 1 et 2 pour l'interface.

Ce qui est attendu de votre part

Aucun schéma de base de données ni squelette de code ne vous est fourni. Vous concevez vous-mêmes :

- Le **modèle de données** (tables, colonnes, types, relations) à partir des besoins fonctionnels décrits ci-dessous ;
- L'**architecture du backend** (organisation des fichiers, routes, controllers, middlewares) ;
- L'**interface frontend** (pages, formulaires, navigation) consommant votre propre API.

Ce projet évalue votre capacité à transformer un besoin exprimé en langage naturel en un système fonctionnel complet, sur une durée proche de celle d'une mission réelle en entreprise.

Besoins fonctionnels (cahier des charges)

1. Gestion des auteurs

- Consulter la liste des auteurs enregistrés.
- Ajouter, modifier ou supprimer un auteur (nom, nationalité).

2. Gestion des adhérents

- Consulter la liste des adhérents inscrits à la bibliothèque (nom, contact).
- Ajouter, modifier ou supprimer un adhérent.
- Consulter, pour un adhérent donné, l'historique de ses emprunts (en cours et passés).

3. Gestion des livres

- Consulter la liste des livres, avec le nom de l'auteur associé à chaque livre.
- Ajouter, modifier ou supprimer un livre (titre, auteur, année de publication).
- Chaque livre a un statut de disponibilité (disponible / emprunté), visible dans la liste.
- Rechercher un livre par titre ou par nom d'auteur, avec un affichage paginé si la liste est longue.

4. Gestion des emprunts

- Enregistrer un emprunt : adhérent, livre concerné, date de retour prévue.
- Un emprunt ne peut pas être créé si le livre est déjà indiqué comme emprunté.
- Lorsqu'un emprunt est créé, le livre concerné passe automatiquement au statut « emprunté ».
- Enregistrer le retour d'un livre : le livre redevient « disponible ».
- Consulter la liste des emprunts en cours et la liste des emprunts **en retard** (date de retour prévue dépassée et livre non rendu).

5. Statistiques & tableau de bord

- Nombre total de livres, d'adhérents, d'emprunts en cours et d'emprunts en retard.
- Le livre le plus emprunté et l'adhérent le plus actif (nombre d'emprunts).

6. Interface (frontend)

- Navigation claire entre les sections : Livres, Adhérents, Emprunts, Tableau de bord.
- Formulaire pour ajouter un livre, un auteur, un adhérent et un emprunt, avec messages d'erreur clairs (ex. livre indisponible).
- Une vue distinguant visuellement les emprunts en cours et les emprunts en retard (ex. couleur différente).
- Une page ou section affichant le tableau de bord / statistiques.
- L'interface communique avec l'API via `fetch()`.

Livrables attendus

Le rendu se fait sous forme d'un dépôt GitHub unique contenant les éléments suivants :

Élément	Détail
Diagramme entité-relation	Schéma de votre modèle de données (image ou diagramme), avec tables, clés et relations.
Script SQL	Fichier <code>schema.sql</code> permettant de recréer vos tables depuis zéro.
Backend	API Express connectée à PostgreSQL, organisée en routes / controllers, avec middlewares de logging, validation et gestion d'erreurs.
Frontend	Pages HTML/CSS/JS consommant l'API et couvrant l'ensemble des besoins fonctionnels listés ci-dessus.
README	Instructions d'installation, <code>.env.example</code> , et description rapide des choix de modélisation.

Aucun code de départ n'est fourni : la structure des dossiers (routes, controllers, middlewares, fichiers frontend) est à votre initiative.

Planning indicatif sur les deux semaines

Semaine 1 (S14) – Conception & Backend

Jour	Objectif
Lundi	Analyse des besoins, conception du modèle de données (y compris adhérents), rédaction du script SQL, connexion PostgreSQL.
Mardi	Routes/controllers pour Auteurs et Adhérents (CRUD complet).
Mercredi	Routes/controllers pour Livres : CRUD, recherche, pagination.
Jeudi	Logique métier des Emprunts : création, retour, détection des retards, liaison avec les adhérents.
Vendredi	Middlewares (logger, validation, gestion d'erreurs), route Statistiques, tests complets via Postman. Jalon intermédiaire.

Jalon intermédiaire – fin de semaine 1

Une courte démonstration (5 minutes, via Postman) de l'API fonctionnelle est présentée à la fin de la semaine 1 : CRUD complet, création/retour d'un emprunt, détection d'un retard. Ce jalon conditionne le passage à la phase frontend.

Semaine 2 (S15) – Frontend & finalisation

Jour	Objectif
Lundi	Architecture frontend, pages de base, affichage des livres et adhérents via l'API (lecture).
Mardi	Formulaires (ajout livre, auteur, adhérent, emprunt) et gestion des messages d'erreur côté interface.
Mercredi	Vue des emprunts en cours / en retard, action de retour d'un livre.
Jeudi	Tableau de bord / statistiques, recherche et pagination côté interface.
Vendredi	Tests de bout en bout, finalisation du README, présentation.

Critères d'évaluation

Critère	Détail	Poids
Modélisation des données	Pertinence du schéma, des relations (y compris adhérents), cohérence avec les besoins.	15%
Backend – fonctionnement	CRUD complet, logique métier des emprunts, requêtes SQL sécurisées, route statistiques.	25%

Critère	Détail	Poids
Backend – structure & middlewares	Séparation routes/controllers, logger, validation, gestion d'erreurs centralisée.	10%
Frontend	Couverture fonctionnelle complète, communication correcte avec l'API, clarté de l'interface.	25%
Logique métier avancée	Disponibilité des livres, détection des retards, statistiques correctes.	15%
Jalon intermédiaire & documentation	Démonstration backend tenue en fin de semaine 1, README clair, diagramme ER lisible.	10%

Pistes bonus (facultatif)

- Système de réservation d'un livre déjà emprunté (file d'attente).
- Export CSV de la liste des emprunts en retard.
- Notifications visuelles (toast) après chaque action réussie côté frontend.
- Filtrage des livres par disponibilité ou par auteur depuis l'interface.

Modalités de rendu

- Dépôt GitHub individuel, lien à transmettre selon les modalités communiquées en cours.
- Le respect des besoins fonctionnels prime sur la sophistication technique : un projet simple mais entièrement fonctionnel est mieux noté qu'un projet ambitieux mais incomplet.